Rapport pour le projet : Développez une preuve de concept

Thomas Durand-Texte*

July 27, 2023

Abstract

Ce document vise à présenter le travail réalisé pour le projet "Développez une preuve de concept" dans le cadre de la formation Ingénieur Machine Learning d'OpenClassrooms. Le contexte et l'intérêt des méthodes développées dans les articles de la littérature sont présentés. La méthode "baseline" est la corrélation d'images numérique (communément appelée DIC en anglais). Celle-ci est brievement abordée, et est utilisée comme référence afin d'estimer l'apport de la méthode proposée. Les articles [1, 2] décrivant les modèles "Deep-DIC" et "DIC-Net" et sont utilisés comme base de travail. Toutefois, plusieurs modifications ont été apportées et améliorent significativement les résultats.

Contents

1 Contexte **1**

2 Génération du dataset **2**
2.1 Interpolation des images 2
2.2 Protocole pour une paire d'images . . 2
2.3 Génération d'images de mouchetis . . 3
2.4 Image de référence 3
2.5 Image déformée 4

3 Corrélation d'images numériques **4**

4 Modèle Deep Learning **5**
4.1 Architecture 5
4.2 Hyper-paramètres 6
4.3 Modifications du réseau 6

5 Comparaisons des méthodes **8**
5.1 Images simulées 8
5.2 Images réelles 8

6 Conclusion **9**

A Résultats supplémentaires **10**

1 Contexte

Depuis les années 1980, la corrélation d'images numériques (DIC, Digital Image correlation) a connu un fort développement. Elle est actuellement une méthode de référence pour les mesures de forme et de déformations de surface [3]. Le principe utilisé est de minimiser la différence entre une image de référence et une image de la surface déformée en estimant de manière itérative les paramètres d'une fonction qui permet de passer des coordonnées d'une image à l'autre. Ce processus peut être effectué sur des sous-images (ou *subsets*, *e.g.* 15×15 pixels) avec une fonction polynomiale, ou alors sur l'image (quasi-)complète en estimant le déplacement de points de contrôle d'un maillage.

Bien que ce groupe de méthodes ait été largement développé et commercialisé par différentes entreprises, des limitations persistent :

1. la taille des subsets et la fonction de forme sont difficiles à optimiser ;
2. les effets de bords réduisent la précision près des limites de la surface observée ou proche de fissures ;
3. la précision est limitée pour certains cas, notamment pour l'estimation d'efforts.

Différents travaux de recherche ont été réalisés ces dernières années pour appliquer les réseaux de neurones convolutionnels (CNN) à la corrélation d'images : [4, 5, 1, 2, 6, 7, 8].

Les deux articles sur lesquels le présent travail est basé sont [1] et [2]. Les deux modèles utilisent une architecture de type U-Net [9]. Pour "Deep-DIC", le dataset est composé d'images synthétiques et les données cibles, *i.e.* déplacements et déformations, correspondent à un mouvement de corps rigide et une gaussienne. Pour "DIC-Net", le dataset est partiellement composé d'images réelles et les déformées sont calculées avec des éléments finis de Hermite. Dans les deux cas, les images initiales sont utilisées pour générer des paires d'images, une image de référence et une image déformée, à partir des déformations (aléatoires) cibles. De plus, afin de limiter la redondance des informations locales, les images sont rognées aléatoirement pour obtenir une paire d'images de taille 128×128 pixels.

*Corresponding author: thomas.durandtexte@protonmail.com
Date: July 27, 2023

Le dataset et les images originales de l'article [2] étant stockés sur Baidu Netdisk, le choix ici est de générer un dataset à partir d'images numériques, à l'instar de Yang *et al.* [1] en utilsant le protocle de Wang *et al.* [2], à savoir la génération d'images déformées avec des éléments de Hermite.

Des codes sont disponibles sur `https://github.com/YinWang20/DIC-Net` et sont utilisés comme base, mais des modifications ont été apportées. Les codes à disposition sont notamment en Matlab® et ont été convertis / implémentés en Python.

La Section 2 présente la méthode utilisée pour générer le dataset. La Section 3 aborde le principe de la corrélation d'images numériques. La Section 4 explique le modèle de base et les améliorations apportées, et, finalement, les résultats de différentes méthodes sont comparés dans la Section 5.

2 Génération du dataset

Le dataset est composé de paires d'images : une image de référence et une image dite déformée. L'objectif du modèle est d'estimer le déplacement de chaque pixel entre l'image de référence et l'image déformée, ainsi que les dérivées spatiales du déplacement.

Afin de générer une paire d'image, une première image d'un mouchetis est requise. Celle-ci va être interpolée pour créer une paire d'image. Afin de diminuer les temps de calcul requis, l'implémentation du dataset est réalisée avec deux paires de datasets / dataloaders :

1. génération et stockage des images initiales ;
2. transformation des images initiales et calcul de la paire d'image.

Toutes les opérations sont réalisées dans la mémoire vive, donc aucune image n'est enregistrée. De plus, contrairement aux méthodes précédentes, les réseaux de neurones voient toujours de nouvelles images.

2.1 Interpolation des images

Le choix de la méthode d'interpolation est crucial pour la qualité des résultats pour les méthodes de corrélation d'images [3]. Ainsi, il semble cohérent de bien choisir la méthode d'interpolation pour générer et interpoler les images du dataset utilisé afin d'entrainer un réseau de neurones visant à estimer les déplacements et déformations de paires d'images.

La réponse à une impulsion pour différentes méthodes d'interpolation est affichée sur la Figure 1. Une interpolation bilinéaire induit une interpolation linéaire entre deux pixels. L'interpolation bicubique semble fournir des résultats plus intéressants, mais un rebond négatif est visible ; cette méthode ne fait pas partie des méthodes préconisées pour la corrélation d'images.

Pour la corrélation d'images, habituellement, un noyau d'interpolation est pré-calculé à chaque position d'interpolation (de -0,5 à 0.5 pixels, avec un pas défini). Ces noyaux sont optimisés à partir d'une réponse fréquentielle choisie. Ici, les noyaux sont directement calculés à partir de la réponse spatiale choisie, *i.e.* une gaussienne, ayant une réponse fréquentielle connue et valide pour la corrélation d'images.

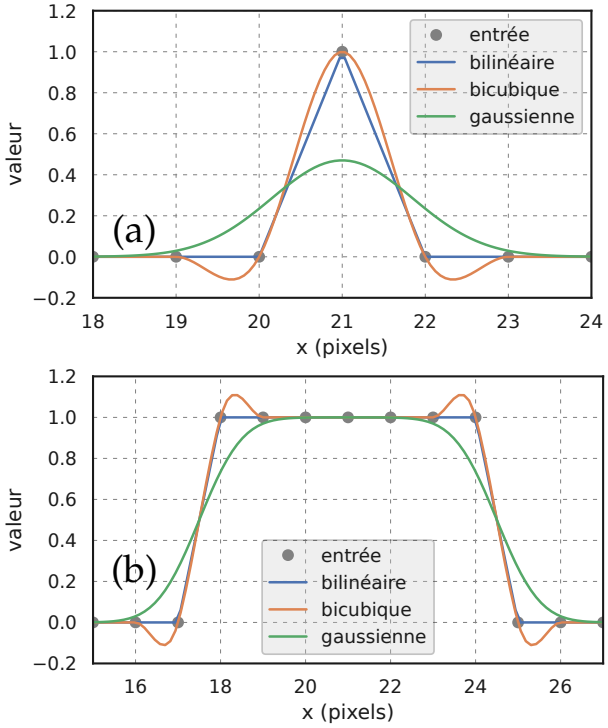

Figure 1: *Exemple de réponse impulsionnelle pour différentes méthodes d'interpolation.*

À noter que les méthodes d'interpolation basées sur une réponse impulsionnelle, contraitement à celles basées sur une approche polynomiale, induisent un filtrage plus conséquent et une potentielle variation de niveaux. Ceci n'est pas génant puisque les images sont normalisées et que l'image de référence et l'image déformée sont interpolées avec la même méthode.

2.2 Protocole pour une paire d'images

Le protocole de génération d'une paire d'image est affiché Figure 2.

Chaque image initiale est transformée afin de

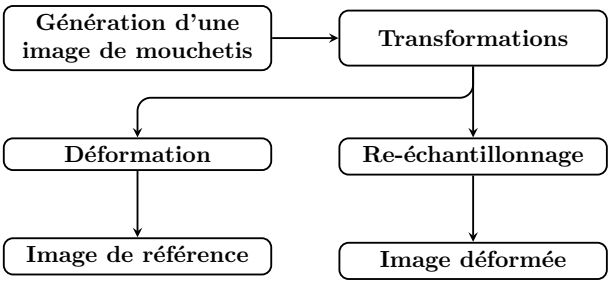

Figure 2: *Protocole pour générer une paire d'image*

générer plus images avec moins de ressources. Les trois transformations suivantes sont appliquées :

- négatif de l'image ($\times 2$)
- miroir horizontal ($\times 2$)
- rotation de l'image de 90° ($\times 4$)

En combinant les transformations, au total 16 images sont générées à partir d'une première image.

2.3 Génération d'images de mouchetis

Afin d'apairer les pixels d'une image à une autre, il est nécessaire d'avoir une information spatiale dense. Pour ce faire, un motif aléatoire est réalisé. Habituellement, des bombes de peintures sont utilisées, en ayant au préalable élargi la buse d'expulsion, mais on peut aussi utiliser des pochoirs collés avec de la colle temporaire, type post-it, si l'on a besoin de motifs plus larges pour des surfaces de mesure relativement grandes.

Pour le dataset de ce projet, la méthode présentée par le papier Deep-DIC est utilisée, en y apportant quelques motifications :

- changement de la méthode d'interpolation (*cf. supra*)
- ajout d'une rotation aléatoire des ellipses
- modification de la taille des ellipses

En effet, pour la corrélation d'images, la taille des taches est censée être comprise entre 3 et 6 pixels environ afin de maximiser les performances [3], et ce du fait du processus d'enregistrement des images par les caméras et afin d'obtenir un gradient correct dans les images. La taille initiale des ellipses est ainsi divisée par deux, comparativement aux équations présentées dans [1].

Une étape supplémentaire a aussi été ajoutée par rapport à l'algorithme de Deep-DIC : l'image est générée avec une résolution 4 fois supérieure (16 fois plus de pixels) puis un filtre convolutionnel moyenneur de dimension 4×4 est appliqué, avant d'appliquer un filtre gaussien. Cette étape est intéressante pour simuler l'intégration spatiale réalisée par les capteurs d'une caméra. L'algorithme utilisé pour générer un mouchetis est le suivant :

1. initialiser une image noire (4*128 $\times$ 4*128 pixels)
2. générer N ellipses aléatoirement
3. ajouter séquentiellement les ellipses à l'image
4. appliquer un filtre convolutionnel moyenneur de taille et de pas égaux à 4
5. appliquer un filtre gaussien

Un exemple d'image générée est affiché sur la Figure 3.

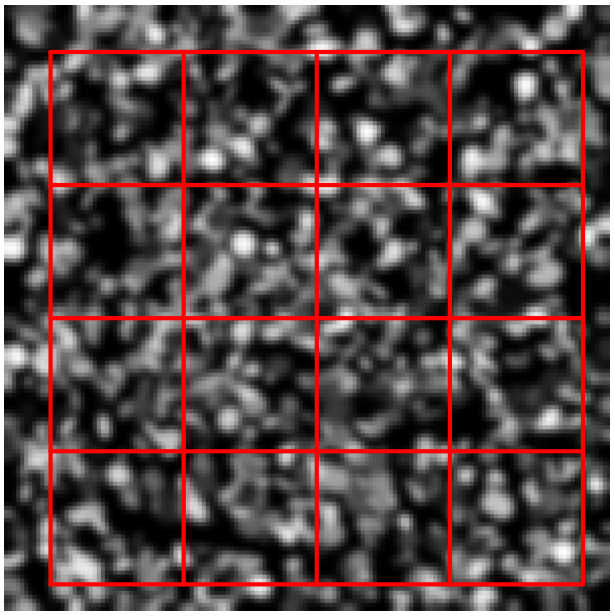

Figure 3: *Exemple d'image de mouchetis générée, les lignes rouges représente un maillage avec des éléments de taille 33 $\times$ 33 pixels.*

2.4 Image de référence

Afin de simplifier la mise en équation du problème, l'image de référence est calculée en interpolant l'image avec l'algorithme suivant:

1. choix aléatoire d'une taille de maillage (5, 9, 17, 33 ou 65 pixels, voir Figure 3 pour un exemple à 33 pixels) ;
2. génération aléatoire des déplacements et déformations ;
3. calcul des positions dans l'image initiale de chaque pixel $(u^{def}, v^{def}) = (u^{ref} + \Delta u,\ v^{ref} + \Delta v)$;
4. interpolation de l'image initiale aux positions (u^{def}, v^{def}).

À noter que, ici, les positions (u^{ref}, v^{ref}) correspondent à un maille de 128 $\times$ 128 pixels (0 $\rightarrow$ 127, 0 $\rightarrow$ 127), et que les déplacements $(\Delta u, \Delta v)$ et les déformations $(\partial\Delta u/\partial u, \partial\Delta u/\partial v, \partial\Delta v/\partial u, \partial\Delta v/\partial v)$ sont calculés à l'aide d'un maillage d'élements finis de Hermite (voir [2] pour les détails de calcul).

2.5 Image déformée

Pour obtenir l'image dite déformée, un simple ré-échantillonnage est réalisé en fonction des positions (u^{def}, v^{def}), et ce afin que la totalité de l'information de l'image de référence soit contenue dans l'image déformée. Les étapes de calcul sont :

1. calcul des valeurs minimales et maximales de u^{def} et v^{def}, *floor* et *ceil* $\rightarrow (u_0, u_1, v_0, v_1)$;
2. génération d'un maillage de 128×128 pixels $(u_0 \rightarrow u_1,\ v_0 \rightarrow v_1)$;
3. interpolation de l'image initiale aux points du maillage ;

avec (u_0, u_1) encadrant $(\min(u^{def}),\ \max(u^{def}))$, et (v_0, v_1) encadrant $(\min(v^{def}),\ \max(v^{def}))$.

Cette opération de ré-échantillonnage va cependant modifier les valeurs de déplacements et déformations. Il est donc nécessaire de recalculer les valeurs correspondantes dans le repère de l'image initiale.

Pour une position (u^{rs}, v^{rs}) dans le repère de l'image ré-echantillonnée, la position correspondante dans le repère initial est $(u_0 + u^{rs} * \alpha_u,\ v_0 + v^{rs} * \alpha_v)$, avec α_u et α_v les pas spatiaux de l'image ré-échantillonnée dans le repère initial. En conséquence, pour passer du repère initial au repère ré-échantillonné, on a :

$$\begin{pmatrix} u^{rs} \\ v^{rs} \end{pmatrix} = \begin{pmatrix} (u^{ref} + \Delta u - u_0)/\alpha_u \\ (v^{ref} + \Delta v - v_0)/\alpha_v \end{pmatrix} \quad . \tag{1}$$

Les valeurs de déplacement et déformées dans le repère ré-échantillonné par rapport aux positions (u^{ref}, v^{ref}) sont obtenues avec :

$$\begin{pmatrix} \Delta u^{rs} \\ \Delta v^{rs} \\ \epsilon^{rs}_{xx} \\ \epsilon^{rs}_{xy} \\ \epsilon^{rs}_{yx} \\ \epsilon^{rs}_{yy} \end{pmatrix} = \begin{pmatrix} u^{rs} - u^{ref} \\ v^{rs} - v^{ref} \\ \partial \Delta u^{rs}/\partial u^{ref} \\ \partial \Delta u^{rs}/\partial v^{ref} \\ \partial \Delta v^{rs}/\partial u^{ref} \\ \partial \Delta v^{rs}/\partial v^{ref} \end{pmatrix} \tag{2}$$

$$\begin{pmatrix} \Delta u^{rs} \\ \Delta v^{rs} \\ \epsilon^{rs}_{xx} \\ \epsilon^{rs}_{xy} \\ \epsilon^{rs}_{yx} \\ \epsilon^{rs}_{yy} \end{pmatrix} = \begin{pmatrix} (u^{ref}(1-\alpha_u) + \Delta u - u_0)/\alpha_u \\ (v^{ref}(1-\alpha_v) + \Delta v - v_0)/\alpha_v \\ (1 - \alpha_u + (\partial \Delta u/\partial u^{ref}))/\alpha_u \\ (\partial \Delta u/\partial v^{ref})/\alpha_u \\ (\partial \Delta v/\partial u^{ref})/\alpha_v \\ (1 - \alpha_v + (\partial \Delta v/\partial v^{ref}))/\alpha_v \end{pmatrix} \tag{3}$$

Ainsi, pour obtenir les valeurs recherchées :

$$\begin{pmatrix} \Delta u \\ \Delta v \\ \epsilon_{xx} \\ \epsilon_{xy} \\ \epsilon_{yx} \\ \epsilon_{yy} \end{pmatrix} = \begin{pmatrix} u^{def} - u^{ref} \\ v^{def} - v^{ref} \\ \partial \Delta u/\partial u^{ref} \\ \partial \Delta u/\partial v^{ref} \\ \partial \Delta v/\partial u^{ref} \\ \partial \Delta v/\partial v^{ref} \end{pmatrix} \quad , \tag{4}$$

il faut alors utiliser :

$$\begin{pmatrix} \Delta u \\ \Delta v \\ \epsilon_{xx} \\ \epsilon_{xy} \\ \epsilon_{yx} \\ \epsilon_{yy} \end{pmatrix} = \begin{pmatrix} \alpha_u \Delta u^{rs} - u^{ref}(1-\alpha_u) + u_0 \\ \alpha_v \Delta v^{rs} - v^{ref}(1-\alpha_v) + v_0 \\ \alpha_u \epsilon^{rs}_{xx} - (1-\alpha_u) \\ \alpha_u \epsilon^{rs}_{xy} \\ \alpha_v \epsilon^{rs}_{yx} \\ \alpha_v \epsilon^{rs}_{yy} - (1-\alpha_v) \end{pmatrix} \quad . \tag{5}$$

Le modèle réalisé intègre directement cette transformation. Il prend donc en entrée :

- la paire d'images ;
- les positions (u^{ref}, v^{ref}) ;
- les informations de ré-échantillonnage (u_0, v_0, α_u, α_v).

Un exemple de paire d'images et de déplacements associés est affiché sur la Figure 4. Les déformations correspondent aux dérivées spatiales des déplacements Δu et Δv (*cf.* Eq. (4)).

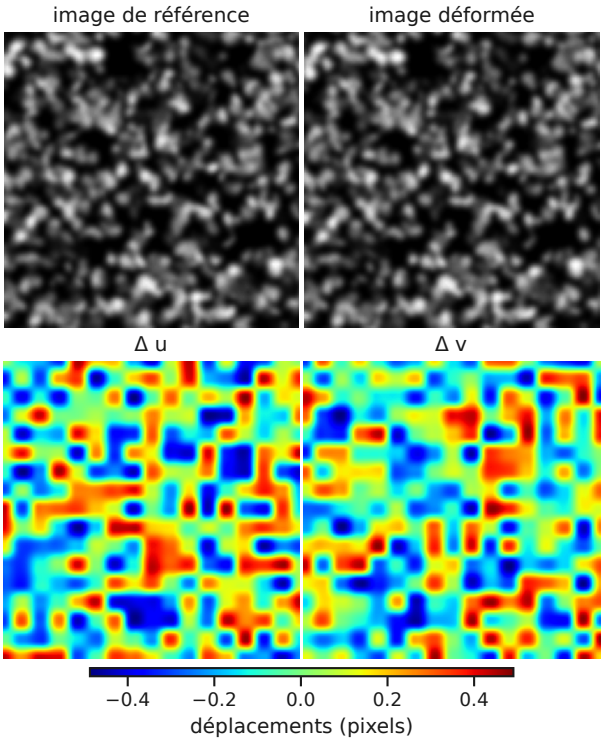

Figure 4: *Exemple de paire d'images et de déplacements (Δu, Δv) avec un maillage de 9×9 pixels.*

3 Corrélation d'images numériques

La corrélation d'images numériques est une méthode largement utilisée depuis plus de 40 ans. Il s'agit d'une des mesures de référence pour la recherche et l'industrie [3]. Cette section présente le principe de base de la méthode. En ignorant les distortions

optiques, sa mise en oeuvre est relativement aisée et rapide.

À noter que, pour simplifier les algorithmes, les variations d'illumination ne sont pas prises en compte puisque les images sont normalisées et que les variations locales d'illumination ne sont pas simulées. Une amélioriation possible du dataset est donc de simuler des variations locales d'illumination.

Le principe de l'estimation du décalage entre deux images est présenté sur la Figure 5 pour un cas 1D. À partir du gradient de l'image de référence ($\partial F(x)/\partial x$) et de la différence entre les deux images ($\Delta I = G(x) - F(x)$), le décalage Δx est retrouvé avec :

$$\Delta x = -\frac{\Delta I}{\partial F(x)/\partial x} \tag{6}$$

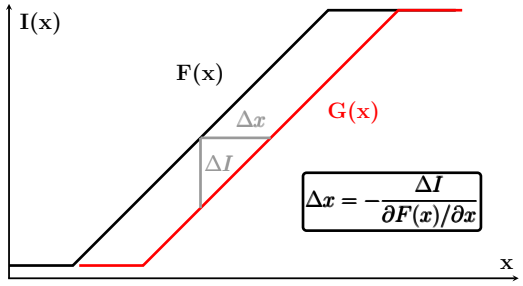

Figure 5: *Principe de la corrélation d'image pour le cas 1D*

Lorsque l'on traite des images, l'estimation de Δx est effectuée sur une petite surface (*subset* en anglais), avec une approximation (le plus souvent) polynomiale d'ordre 2 : $\Delta x(u,v) = p_0 + p_1 u + p_2 v + p_3 u^2 + p_4 uv + p_5 v^2$. Deux polynomes sont alors utilisés, p_u pour Δu et p_v pour Δv.

Les deux polynomes sont estimés de manière itérative à l'aide d'une estimation par moindres carrés. En définissant une base de fonction $\phi_i(u,v)$, (1, u, v, u^2, etc...), l'estimateur est construit tel que:

$$M = [A^T A]^{-1} A^T \tag{7}$$

avec

$$A = \left[\frac{\Phi(u,v) * F_u(u,v)}{\Phi(u,v) * F_v(u,v)} \right] \quad , \tag{8}$$

$\Phi(u,v)$ étant une matrice à six lignes ($\phi_0(u,v)$, $\phi_1(u,v)$, ...), $F_u(u,v)$ et $F_v(u,v)$ étant les dérivées spatialles de $F(u,v)$ selon les axes u et v respectivement. À noter que chaque ligne de $\Phi(u,v)$ est multipliée par $F_{u/v}(u,v)$: $\Phi(u,v) * F_u(u,v)$ n'est donc pas une opération matricielle.

En partant d'une position suffisamment proche de la solution, les polynomes p_u et p_v sont obtenus avec l'opération matricielle :

$$\begin{pmatrix} \Delta p_u \\ \Delta p_v \end{pmatrix} = M(F(u,v) - G(u,v)) \quad . \tag{9}$$

Avec une approche itérative, p_u et p_v sont estimés avec l'algorithme :

1. calculer (u_{def}, v_{def}) = f(p_u^i, p_v^i, u, v) ;
2. interpoler $G(u_{def}, v_{def})$;
3. estimer Δp_u et Δp_v ;
4. estimer p_u^{i+1} et p_v^{i+1} par moindres carrés tel que f(p_u^{i+1}, p_v^{i+1}, u, v) $\approx$ f(p_u^i, p_v^i,f(Δp_u, Δp_v, u, v)).

Un example d'estimation est affiché sur la Figure 6. La déformation estimée est visiblement d'ordre 2.

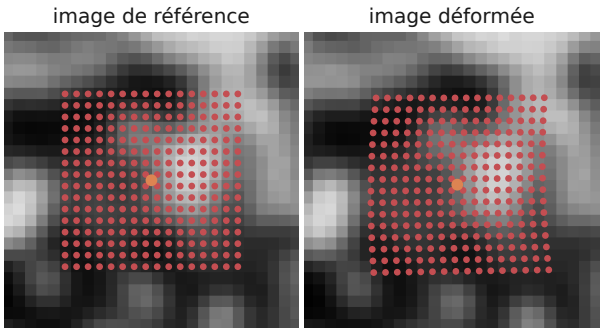

Figure 6: *Example d'estimation de déformation : les points rouges correspondent aux positions des pixels dans l'image de référence (initialisation à gauche) et dans l'image déformée (estimation à droite).*

4 Modèle Deep Learning

4.1 Architecture

L'architecture générale proposée pour DIC-Net est affichée sur la Figure 7. Celle-ci est basée sur l'architecture U-Net [9]. Une première convolution 2D de 64 filtres est utilisée pour comparer la paire d'images. Puis, une suite de convolutions par six blocs de modules résiduels (voir Figure 8.(a) pour la structure d'une module résiduel), avec 128, 128, 256, 256, 512, 512 et 1024 filtres en sortie respectivement. Les auteurs de DIC-Net ont ensuite proposé de remplacer les convolutions inverses par des blocs de sur-échantillonnage linéaire suivi de deux convolutions 2D (voir Figure 8.(b)) afin de limiter l'effet damier (*cf.* [2]). Les nombres de filtres utilisés pour les doubles convolutions sont 512, 256, 128 et 64 respectivement. Notons que le modèle U-Net, sur lequel l'architecture étudiée est basée, s'appelle ainsi car la forme du schéma bloc fait penser à un "U" : les sorties des convolutions de la plupart des blocs de modules résiduels sont concaténées avec les sorties des doubles convolutions. Finalement, une convolution 2D (3×3) de 64 filtres suivi d'une convolution (1×1) donne en sortie les déplacements (2 variables) ou les déformations (4 variables).

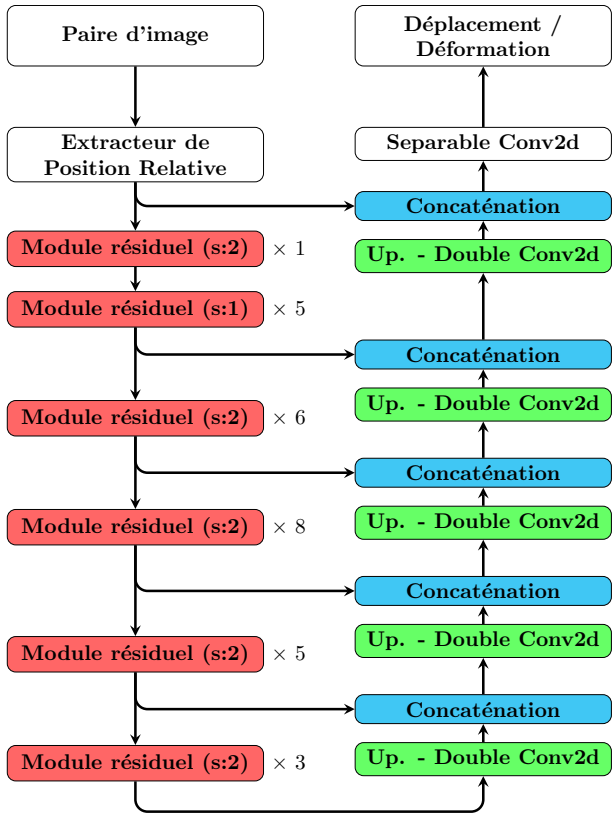

Figure 7: *Schéma bloc de l'architecture du DIC-Net [2], basé sur U-Net [9]*

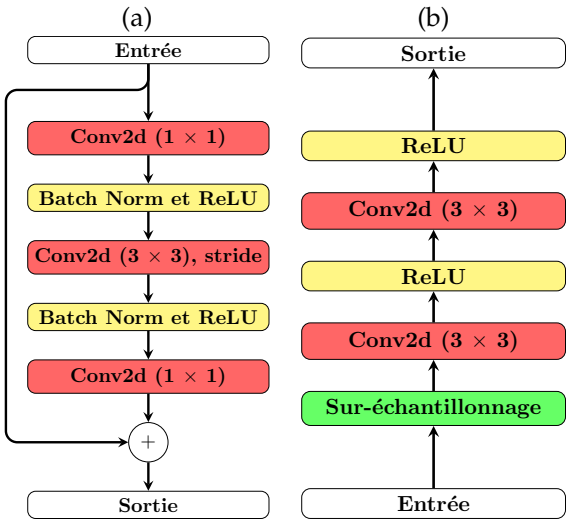

Figure 8: *Schéma bloc de : (a) module résiduel, et (b) sur-échantillonage et double conv2d.*

4.2 Hyper-paramètres

Pour l'entrainement, l'algorithme d'optimisation sélectionné est Adam avec les valeurs par défaut pour les hyper-paramètres : $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$ et un taux de décroissance nul. Toutefois, le taux d'apprentissage est ajusté avec un planificateur. La fonction utlisée est $lr = 10^{-5} * (1 + 9 * 0,94^{epoch})$. La courbe correspondante est affichée sur la Figure 9.

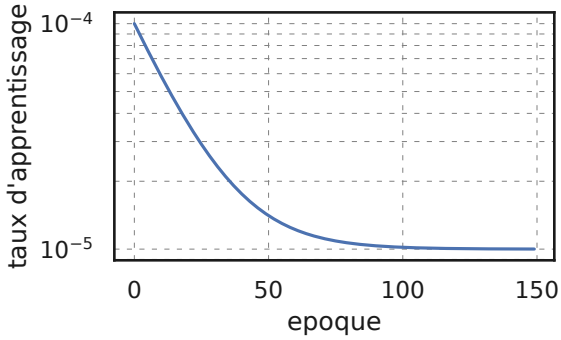

Figure 9: *Courbe du taux d'apprentissage en fonction des epoques.*

4.3 Modifications du réseau

Bien que les résultats annoncés pour Deep-DIC et DIC-Net semblent prometteurs, l'architecture proposée est améliorée, apportant un gain de performance significatif. Les premières modifications, dont les courbes de pertes sont affichées sur la Figure 10, sont :

1. remplacement des Batch Norm par des Group Norm (4 groupes) ;
2. réduction de la taille des *mini-batches* (1 paire d'images au lieu de 16) ;
3. changement de la fonction d'activation de chacun des blocs (ReLU → Mish).

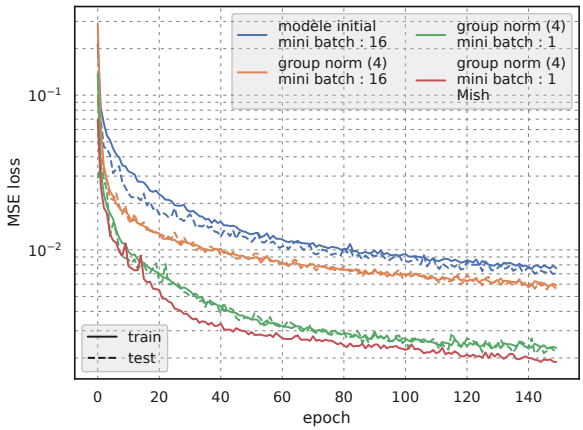

Figure 10: *Courbes de pertes obtenues lors de l'entrainement du modèle pour différentes modifications de normalisations et fonctions d'activation.*

L'utilisation d'une normalisation de groupe plutôt qu'une normalisation de *mini-batch* est très intéressante, puisqu'elle pemet de générer des paramètres

plus robustes aux variations des images en entrée et d'utiliser efficacement des *mini-batches* de taille 1. Comme le montrent les courbes de la Figure 10, le gain est conséquent, puisque les pertes passent de $\approx 0,007$ à $\approx 0,0023$ au bout de 150 epoques. Un autre avantage est aussi la quasi annulation de l'écart entre les courbes obtenues pour le jeu d'entrainement (traits pleins) et le jeu de test (traits discontinus). En pratique, puisque les images sont générées aléatoirement à chaque epoque, les calculs sur un jeu d'entrainement ne sont plus effectués.

D'autres modifications profondes sont aussi effectuées :

1. Échange des blocs de modules résiduels avec des modules de convolution dense ;
2. Remplacement des convolutions 2D par des convolutions séparables augmentées pour les modules de convolution dense et les doubles convolutions ;
3. Création d'un nouveau module pour la première couche de convolution du réseau.

Un bloc de convolution dense est composé d'une suite de couches de convolutions, suivi d'une couche de transition (*cf.* Figure 8.(a)). Les paramètres sont le facteur de croissance k et le nombre de couches. Chaque couche (Figure 8.(b)) est composée d'une convolution (1×1) avec $16 * k$ filtres, suivie d'une convolution (3×3) avec k filtres et d'une normalisation et d'une fonction d'activation (ici Group Norm 4 et Mish). La sortie d'une couche correspond à la concaténation de son entrée avec la sortie de la fonction d'activation. Notons que les convolutions doivent fournir des cartes de dimensions (hauteur et largeur) identiques à l'entrée pour pouvoir effectuer la concaténation. La couche de transition (Figure 8.(c)) est composée d'une convolution (1×1) avec un nombre de filtres égal à la moitié du nombre de canaux en entrée ; si le pas spatial désiré est supérieur à 1, une moyenne spatiale est effectuée (average pooling).

Ici, le facteur de croissance k est choisi égal à 16, et le nombre de couches est tel que le nombre de canaux en sortie de la couche de transition reste identique par rapport à la configuration avec les blocs de modules résiduels (*i.e.* 12, 24, 16, 48, 32 et 96 couches).

Un autre ajout est le remplacement des convolutions (3×3) par des convolutions séparables augmentées (voir Figure 8.(d)). Une couche de convolution séparable est constitué d'une convolution (3×3) avec un groupe égal au nombre de canaux en entrée, suivi d'une convolution (1×1) avec le nombre de canaux désirés en sortie (voir [10] pour plus d'explications). L'ajout effectué ici consiste à utiliser un nombre de filtres $n * N_0$ et un nombre de groupes égal à N_0, induisant que chaque carte est "vue" par n filtres

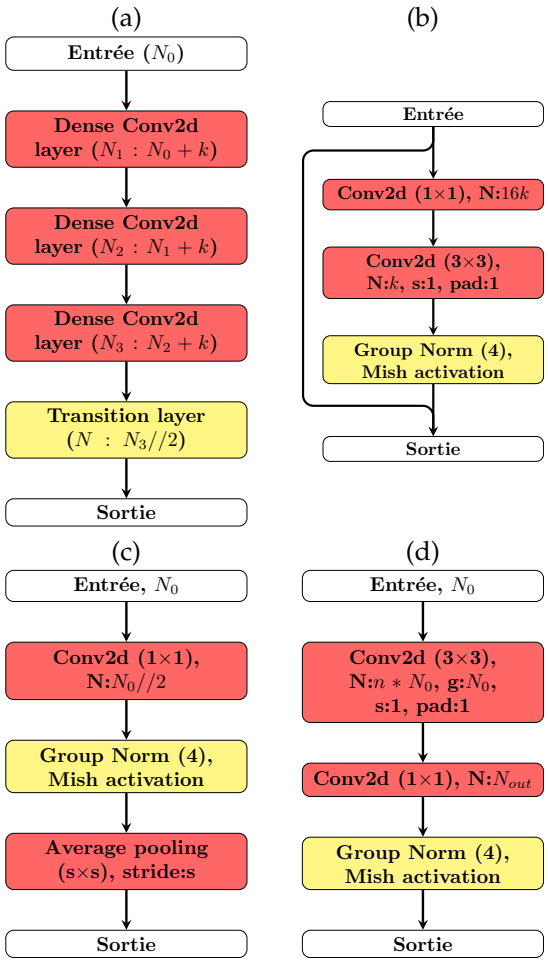

Figure 11: *Schéma bloc de : (a) exemple de module de convolution dense, (b) couche de convolution dense, (c) couche de transition et (d) module de convolution séparable augmentée.*

(3×3) plutôt qu'un seul, d'où l'appellation couche convolutionnelle séparable "augmentée".

Finalement, un nouveau module est proposé pour la première couche de convolution : une convolution séparable intriquée (Figure 12). Le principe est que chaque canal en entrée soit "vu" par les mêmes N_f filtres spatiaux. Les images de référence et déformées ayant des caractéristiques très proches, elle peuvent, *a priori*, être encodées avec les mêmes filtres (3×3). Les $N_f \times C$ cartes résultantes (ici $C = 2$) sont ensuite passées dans un filtre (1×1) pour obtenir N_f cartes en sortie.

Notons que les deux premières modifications augmentent de manière non négligeable le temps de calcul, comme présenté dans le tableau 1. Cependant, comme nous pouvons le voir sur la Figure 13, un gain de performance est encore à noter. La précision étant cruciale pour les applications de corrélation d'images, et les temps de calcul restant tout à fait raisonnables, cet allongement de durée est tout à fait acceptable.

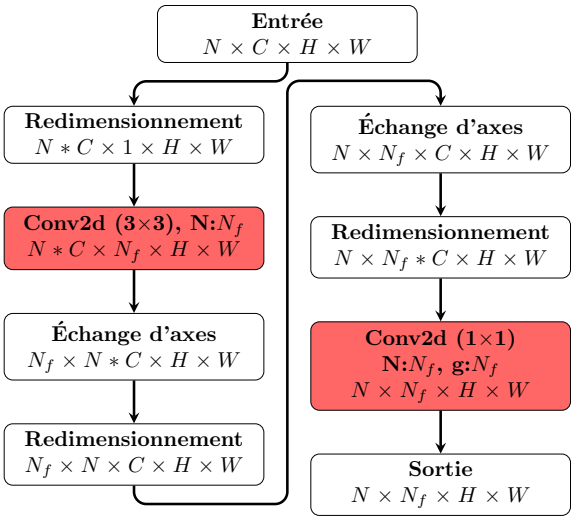

Figure 12: *Schéma bloc du module proposé de convolution séparable intriquée.*

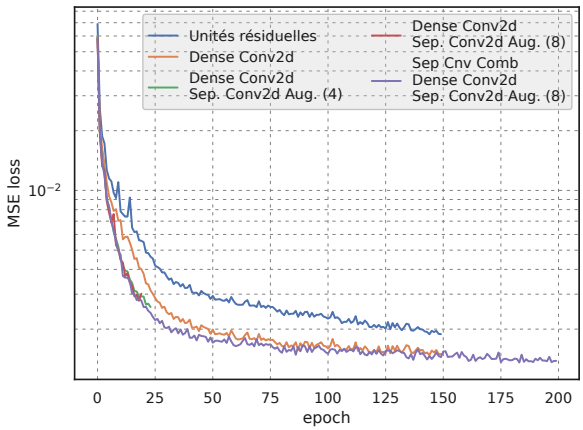

Figure 13: *Courbes de pertes obtenues lors de l'entrainement du modèle pour différentes modifications avec les modules de convolution dense.*

Un autre point intéressant, est que les performances semblent se rejoindre pour le modèle sans et avec convolution séparable augmtentée. Ceci est possiblement du à une limite mathémique / métrologique, liée à la numérisation des images (résolution spatiale et digitalisation sur 8 bits) ainsi qu'à la présence de bruit, inévitable en pratique et simulé dans le dataset proposé.

Modèle	Temps de calcul
Blocs résiduels	5 h 25 min
Dense Conv2d	12 h 54 min
Dense Conv2d + Conv2d Sep. Aug.	13 h 47 min

Table 1: *Temps de calculs approximatifs pour 150 epochs pour les différentes architectures.*

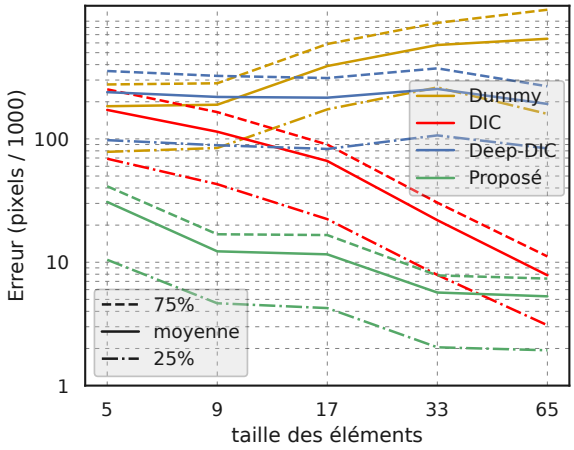

Figure 14: *Erreur en fonction de la taille du maillage.*

5 Comparaisons des méthodes

5.1 Images simulées

Une première comparaison est effectuée avec des images simulées. Une paire unique d'images est utilisée, et des essais supplémentaires seront à effectuer avec plus d'images. Pour la corrélation d'images, des sous-images de 21 × 21 pixels sont utilisées.

L'erreur en fonction de la taille du maillage est affichée sur la Figure 14.

Il est donc plus pertinent de regarder directement les cartes de déplacements et déformations afin de comparer les différentes méthodes. Les déplacements simulés et prédits pour des maillages de taille 5 et 17 pixels sont affichés sur les Figures 15 et 16. Les cartes correspondant aux autres tailles de maillage et les cartes de déformations sont affichées en annexe.

On note que, pour la méthode de génération d'images proposée :

- Deep-DIC ne fournit des résultats utilisables que pour des maillages de taille 33 ou 65 pixels, et aucun résultat exploitable pour les déformations ;
- DIC fournit des résultats intéressants à partir de 17 pixels pour les déplacements et 33 pixels pour les déformations ;
- le réseau proposé fournit de bons résultats avec le dataset proposé, quelque soit la taille du maillage, pour les déplacements et les déformations.

5.2 Images réelles

Les résultats du modèle proposé sont comparés à ceux de la corrélation d'image pour des images

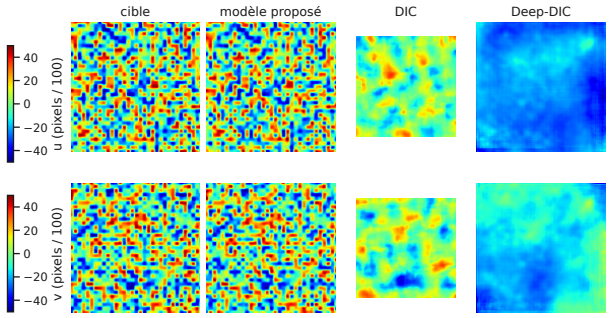

Figure 15: *Déplacements prédits pour une paire d'images avec un maillage de 5 pixels.*

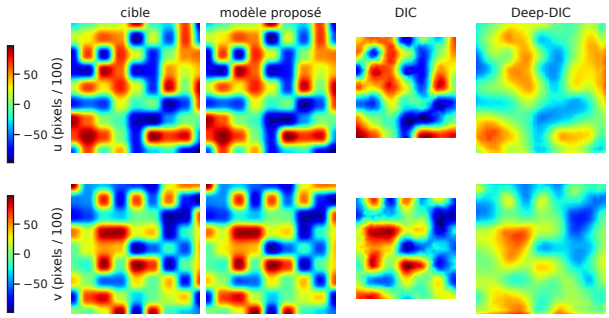

Figure 16: *Déplacements prédits pour une paire d'images avec un maillage de 17 pixels.*

réelles. Les images utilisées font partie d'un jeu de données de mesure utilisé pour l'article [11]. Une poutre est peinte et des ellipses noires sont appliquées à l'aide d'un tampon encreur. Un petit marteau d'impact vient frapper l'arrière de la poutre, et une séquence d'images est enregistrée avec une caméra rapide. La première image de la séquence est affichée Figure 17. On peut noter que le motif est relativement différent de ce que le réseau de neurones a pu voir lors de son entrainement.

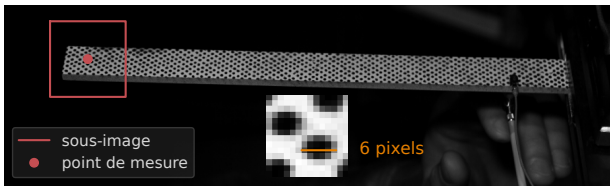

Figure 17: *Image de référence d'une poutre subissant un impact à l'arrière.*

Pour la corrélation d'images, des sous-images de 21×21 pixels sont utilisées. Au total, 20 000 images sont traitées sur une petite zone (en rouge sur la Figure 17). Les résultats pour un point sont affichés sur la Figure 18 dans le domaine temporel (en haut) et dans le domaine spectral (Transformée de Fourier Rapide, en bas). Malgré les différences entre le motif appliqué sur la poutre et le motif du jeu d'entrainement, les résultats sont intéressants pour le réseau entrainé. Toutefois, pour ce cas réel, la corrélation d'images semble fournir des résultats moins bruités. Afin d'obtenir de meilleurs résultats avec le réseau de neurones, un motif plus adapté doit être appliqué à la zone de mesure et le dataset doit être amélioré. À noter aussi que les déplacements sont relativement petits (de l'ordre de 0,05 pixels) comparativement aux déplacements du générés lors de l'entrainement.

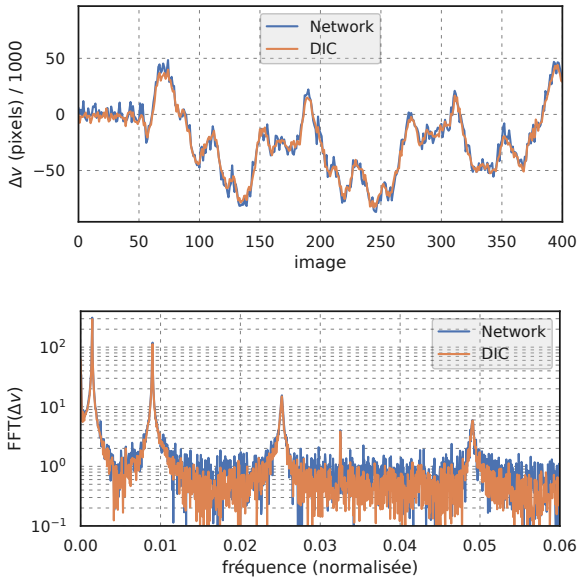

Figure 18: *Signaux (Δv) temporels (haut) et Transformée de Fourier Rapide (bas), mesurés pour la poutre (Figure 17) avec la corrélation d'images et le réseau entrainé.*

6 Conclusion

Ce document présente la démarche réalisée, les algorithmes utilisés et les résultats obtenus pour le projet "Développez une preuve de Concept" dans le cadre de la formation "Ingénieur Machine Learning" d'OpenClassrooms. Un dataset généré numériquement est utilisé pour entrainer différents réseaux de neurones afin de prédire des déformations d'images. Les réseaux proposés dans la littérature sont améliorés, et sont ainsi plus performants que la méthode baseline pour le dataset utilisé. Plusieurs modules sont notamment proposés et testés afin d'améliorer les résultats et/ou réduire le nombre de paramètres du réseau.

Toutefois, l'essai de suivi dans une séquence d'images sur un cas réel n'est que partiellement concluant, puisque les résultats avec la méthode baseline sont moins bruités. Ceci est probablement du au fait

que le motif aléatoire appliqué sur la zone de mesure est assez différent de ce qu'a pu voir le réseau lors de l'entrainement.

Néanmoins, les résultats sont prometteurs, et des améliorations seraient envisageable :

- Amélioration de la génération des paires d'images du dataset proposé :
  - Ajout d'aléas sur la valeur minimale du niveau de gris des taches ;
  - Augmentation de la taille maximale des taches ;
  - Améliorations potentielles de la méthode de génération des déplacements et déformations :
    - Ajout d'aléas du niveau maximal de déplacement ;
    - Ajustement des tailles possibles de maillage par rapport aux objectifs ;
    - Changement de la forme des éléments ;
    - Utilisation d'une autre méthode ;
- Application d'un motif avec une bombe de peinture afin d'obtenir un motif plus proche de celui généré par le dataset d'entrainement ;
- Exploration d'autres architectures ;
- Utilisation de la méthode compound scaling.

References

[1] Ru Yang et al. "Deep DIC: deep learning-based digital image correlation for end-to-end displacement and strain measurement." In: *J Mater Process Technol* 302.117474 (2022).

[2] Yin Wang and Jiaqing Zhao. "DIC-Net: Upgrade the performance of traditional DIC with Hermite dataset and convolution neural network". In: *Optics and Lasers in Engineering* 160 (2023), p. 107278. ISSN: 0143-8166. DOI: https://doi.org/10.1016/j.optlaseng.2022.107278.

[3] Hubert Schreier, Jean-José Orteu, and Michael A. Sutton. *Image Correlation for Shape, Motion and Deformation Measurements*. 1th. Springer New York, NY, 2010. ISBN: 978-1-4419-4622-5. DOI: 10.1007/978-0-387-78747-3.

[4] H.-G. Min et al. "Strain measurement during tensile testing using deep learning-based digital image correlation." In: *Measurement Science and Technology* 31.015014 (2019).

[5] S. Boukhtache et al. "When Deep Learning Meets Digital Image Correlation." In: *Optics and Lasers in Engineering* 136.106308 (2021).

[6] Xiaocen Duan et al. "Digital image correlation based on convolutional neural networks". In: *Optics and Lasers in Engineering* 160 (2023), p. 107234. ISSN: 0143-8166. DOI: https://doi.org/10.1016/j.optlaseng.2022.107234.

[7] S. Boukhtache et al. "A lightweight convolutional neural network as an alternative to DIC to measure in-plane displacement fields". In: *Optics and Lasers in Engineering* 161 (2023), p. 107367. ISSN: 0143-8166. DOI: https://doi.org/10.1016/j.optlaseng.2022.107367.

[8] Shang Jiang et al. "Full-field deformation measurement of structural nodes based on panoramic camera and deep learning-based tracking method". In: *Computers in Industry* 146 (2023), p. 103840. ISSN: 0166-3615. DOI: https://doi.org/10.1016/j.compind.2022.103840.

[9] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *arXiv* (2015). DOI: 10.48550/arXiv.1505.04597.

[10] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. third edition. O'Rreilly Media, Inc., 2023.

[11] P. O'Donoughue et al. "Comparison of three full-field optical measurement techniques applied to vibration analysis". In: *Sci Rep* 13.3261 (2023). DOI: 10.1038/s41598-023-30053-9.

A Résultats supplémentaires

Les cartes ci-dessous correspondent aux déplacements et déformations obtenus pour des images simulées (*cf.* Section 5.1).

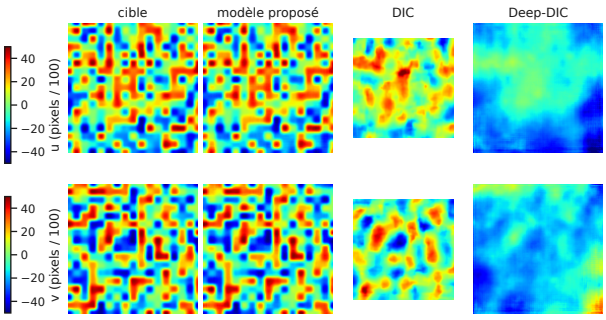

Figure 19: *Déplacements prédits pour une paire d'images avec un maillage de 9 pixels.*

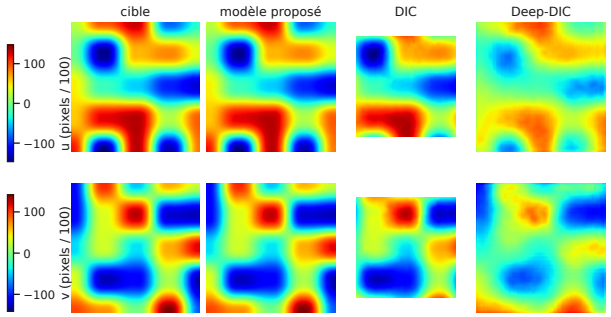

Figure 20: *Déplacements prédits pour une paire d'images avec un maillage de 33 pixels.*

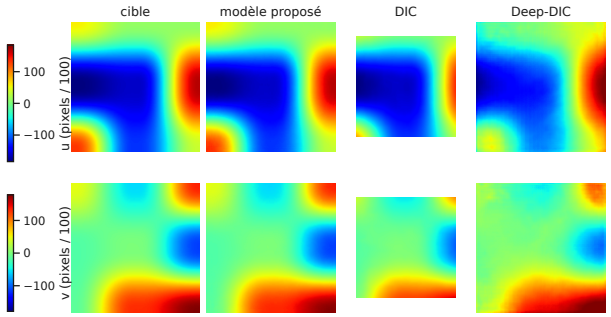

Figure 21: *Déplacements prédits pour une paire d'images avec un maillage de 65 pixels.*

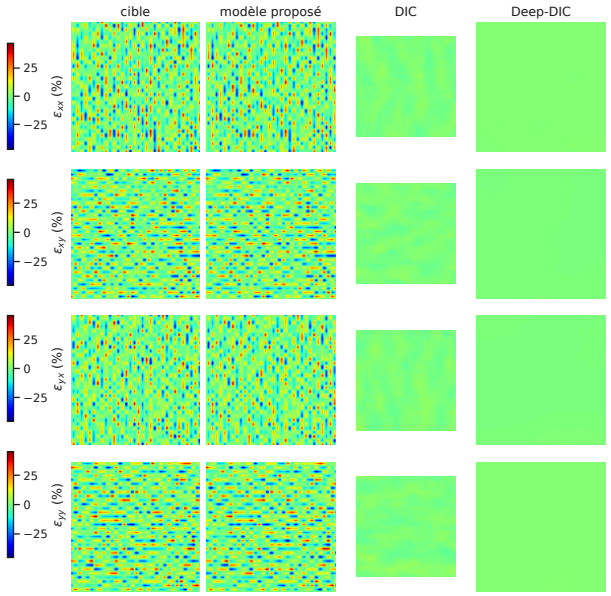

Figure 22: *Déformations prédites pour une paire d'images avec un maillage de 5 pixels.*

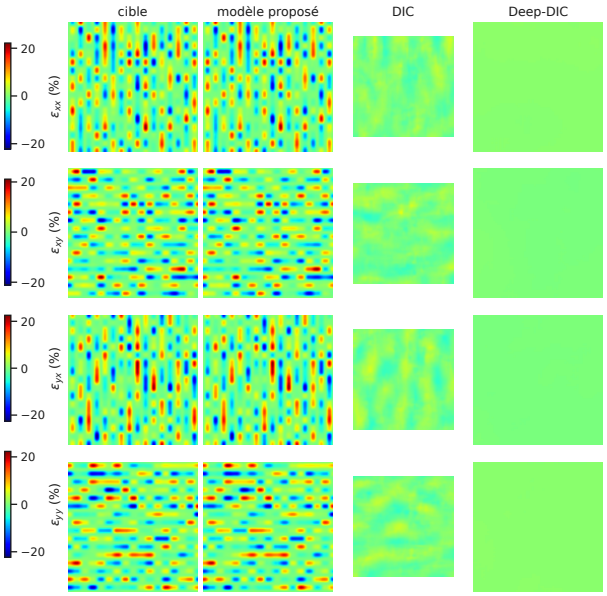

Figure 23: *Déformations prédites pour une paire d'images avec un maillage de 9 pixels.*

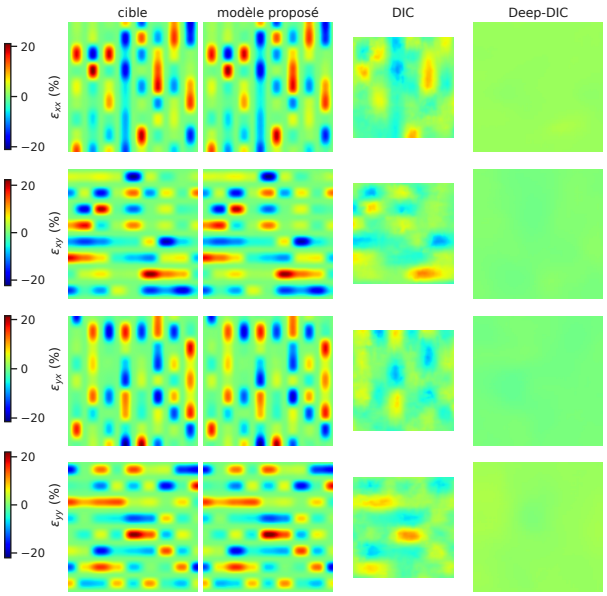

Figure 24: *Déformations prédites pour une paire d'images avec un maillage de 17 pixels.*

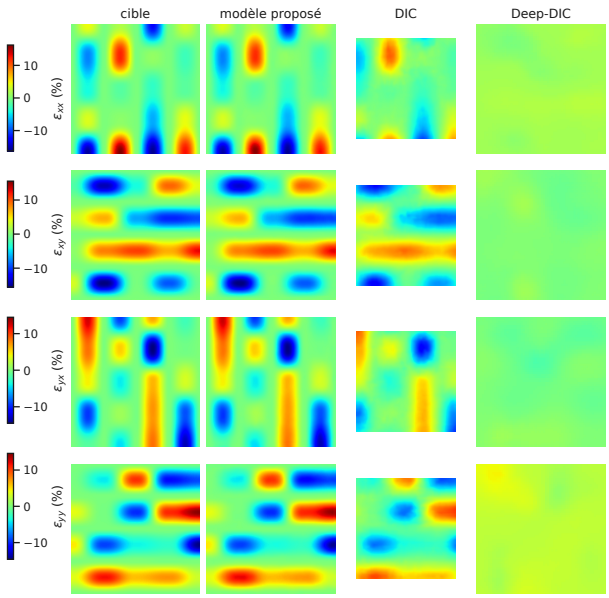

Figure 25: *Déformations prédites pour une paire d'images avec un maillage de 33 pixels.*

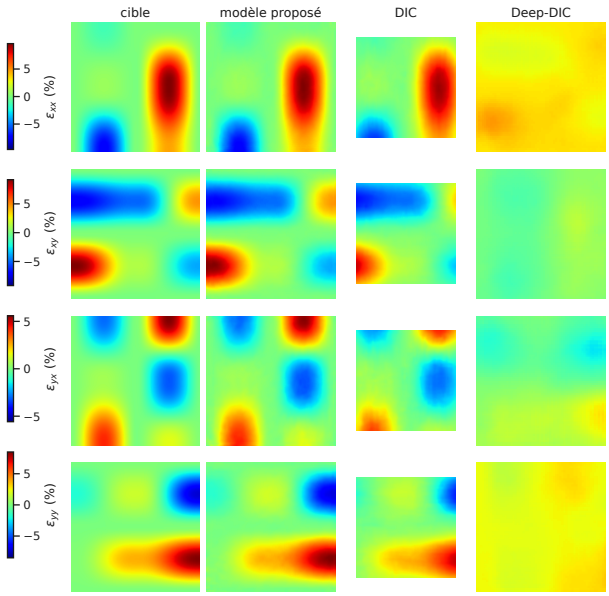

Figure 26: *Déformations prédites pour une paire d'images avec un maillage de 65 pixels.*